

SLEEP STAGE CLASSIFICATION FROM ECG SIGNALS USING DEEP LEARNING

Prepared by:

**Amany Mahmoud Bakry
Shahd Samir Shaaban**

Group 5

Under supervised:

**Dr. Ibrahim Sadek
Eng. Veronicaa William**

Table of Contents

ABSTRACT4

INTRODUCTION4

DATASET5

 Data Source5

 Data Description6

 Number of Samples.....6

 Number of Classes.....6

 Sampling Rate.....6

 Class Distribution Overview.....7

 Train / Validation / Test Split.....7

DATA PREPARATION7

 Import Required Libraries7

 Data Loading and Organization7

 Data Splitting9

 Data Augmentation9

PREPROCESSING10

 Signal Normalization10

 Noise Reduction / Filtering11

 Signal Segmentation12

 Feature Extraction12

 Class Weights13

METHODOLOGY13

 Deep Learning Model Architecture13

 Input Shape Definition13

 Feature Extraction Layers14

Activation Functions	14
Loss Function	14
Optimizer	15
Model Compilation	15
Model Summary	15
TRAINING	16
Number of Epochs	16
Batch Size	16
Learning Rate	16
Validation Strategy	16
Training on Balanced Dataset	17
Callbacks	17
EVALUATION	17
Test Set Evaluation	17
Precision / Recall / F1-score	18
RESULTS	18
Training and Validation Curves	18
Class Distribution	19
Confusion Matrix Visualization	20
ROC Curve	20
DISCUSSION	21
Model Strengths	21
Model Limitations	22
CONCLUSION	22
Study Summary	22
Future Improvements	23

Abstract

- This study proposes a hybrid deep learning framework for automatic sleep stage classification using **electrocardiogram (ECG) signals and heart rate variability (HRV) features**. The dataset consists of multiple subjects with annotated ECG recordings segmented into 30-second epochs and labeled into five sleep stages: Wake (W), N1, N2, N3, and REM.
- The proposed pipeline integrates signal processing, feature engineering, and deep learning. Raw ECG signals are preprocessed using bandpass filtering (0.5–40 Hz) and segmented into fixed-length windows. In parallel, **HRV features** are extracted from each epoch, including time-domain, frequency-domain, and nonlinear descriptors.
- A dual-input neural network architecture is designed, combining a **ResNet1D**-based convolutional backbone for ECG **feature extraction**, a Transformer encoder for temporal dependency modeling, and a fully connected branch for **HRV features**. The extracted representations are fused and passed through dense layers for final classification.
- To improve generalization and handle class imbalance, biologically realistic data augmentation is applied to the training set, along with a composite loss function combining **Focal Loss** and **Label Smoothing**. The model is optimized using the **AdamW** optimizer and trained in a two-phase strategy with **fine-tuning** at a reduced learning rate.
- Experimental results demonstrate strong performance in multi-class sleep stage classification, showing the effectiveness of combining deep signal representations with physiological feature engineering.

Introduction

- Sleep stage classification plays a critical role in **sleep medicine** and **clinical diagnosis**, as it provides valuable insights into sleep quality and disorders. Traditional sleep staging methods rely on manual scoring of **polysomnography (PSG)** signals, which is time-consuming, subjective, and prone to inter-rater variability.
- With the advancement of machine learning and deep learning, automatic sleep stage classification has gained significant attention. In particular, **electrocardiogram (ECG)** signals have emerged as a promising alternative for sleep analysis due to their non-invasive nature and strong correlation with autonomic nervous system activity. **Heart rate**

variability (HRV), derived from ECG signals, further provides meaningful physiological information reflecting sleep dynamics.

- In this study, a **hybrid deep learning approach** is proposed that leverages both raw ECG signals and engineered HRV features. The ECG signals are first preprocessed using bandpass filtering and segmented into fixed **30-second epochs** aligned with sleep stage annotations. HRV features are then extracted to capture complementary information in both time and frequency domains.
- To effectively model the complex temporal dependencies in ECG signals, a **ResNet1D** architecture with dilated convolutions is employed, followed by a Transformer encoder to capture global sequence relationships. In parallel, **HRV features** are processed through a dense neural network branch. The outputs of both branches are fused to perform final sleep stage classification.
- To address class **imbalance** and **improve generalization**, targeted data augmentation is applied to the training set. Furthermore, a hybrid loss function combining Focal Loss and Label Smoothing is used, and the model is optimized using the **AdamW** optimizer in a two-stage training strategy.
- The proposed framework aims to provide an accurate, robust, and automated solution for sleep stage classification using ECG-based signals.

Dataset

Data Source

- The dataset used in this study is derived from the MESA Sleep dataset, which contains physiological signal recordings collected for sleep analysis. **The dataset is organized into two main directories: "Signals" and "Annotations".**
- The **"Signals"** directory contains raw physiological recordings in European Data Format (**.edf**), where each file corresponds to a single subject. These signals include electrocardiogram (**ECG**) and other physiological channels recorded during sleep. Inside the **Signals directory**, there are **10 EDF files** in total, each representing a different subject.



- The "[Annotations](#)" directory contains corresponding annotation files for each subject, providing labeled information about sleep stages over time. Inside the [Annotations directory](#), there are [10 NSRR files](#) corresponding to the same [10 subjects](#).
- **Each subject is represented by a pair of files:**
 - An EDF file containing the raw signal data (e.g., mesa-sleep-0001.edf)
 - An annotation file containing sleep stage labels (e.g., mesa-sleep-0001-nsrr)
- From the available dataset sample, multiple subjects are included, each with long-duration recordings typically spanning several hours of sleep.

Data Description

- **Data type:** Time-series physiological signals ([primarily ECG](#))
- **File format:** EDF ([European Data Format](#))
- **Labels:** Sleep stages ([Wake](#), [N1](#), [N2](#), [N3](#), [REM](#))

Number of Samples

- Each [EDF](#) file represents one subject, and each subject contains multiple signal segments extracted during preprocessing. The total number of samples depends on the segmentation process applied to the continuous recordings.

Number of Classes

- **The classification task is based on five sleep stages:**
 - Wake
 - N1 ([Light Sleep](#))
 - N2
 - N3 ([Deep Sleep](#))
 - REM ([Rapid Eye Movement](#))



Sampling Rate

- The sampling frequency varies depending on the signal channel but is typically consistent within each [EDF file](#). It is extracted programmatically during preprocessing.

Class Distribution

- Sleep stage data is inherently imbalanced, where certain classes such as **N2** dominate the dataset, while others like **N1** and **REM** are less frequent. This imbalance is addressed during training using data augmentation techniques.

Data Split

- The dataset is divided into three subsets:
 - Training set
 - Validation set
 - Test set
- The split is performed at the subject level to avoid data leakage, ensuring that data from the same subject does not appear in multiple subsets.

Data Preparation

Import Required Libraries

- In this study, a comprehensive set of libraries was utilized to support **data processing**, **signal analysis**, **machine learning**, and **deep learning model development**.
- General-purpose libraries such as **NumPy** and **Pandas** were used for **numerical computations** and **data manipulation**. Visualization tools including Matplotlib and Seaborn were employed for plotting and data exploration.
- For signal processing, the **MNE** library was used to **handle physiological signal data in EDF format**, while **SciPy** provided **essential filtering techniques** such as **Butterworth filtering** and **peak detection**.
- Machine learning utilities from **Scikit-learn** were used for **dataset splitting**, **performance evaluation**, and **handling class imbalance**. These included functions for **train-test splitting**, **confusion matrix generation**, **classification reports**, and **class weight computation**.
- Deep learning models were implemented using **TensorFlow** and **Keras**. **Various layers** such as **Conv1D**, **Batch Normalization**, **Dropout**, and **Multi-Head Attention** were used to **build a hybrid neural network architecture**. The **AdamW** optimizer was applied for **efficient training**, along with **callbacks** such as **learning rate reduction** and **model checkpointing**.

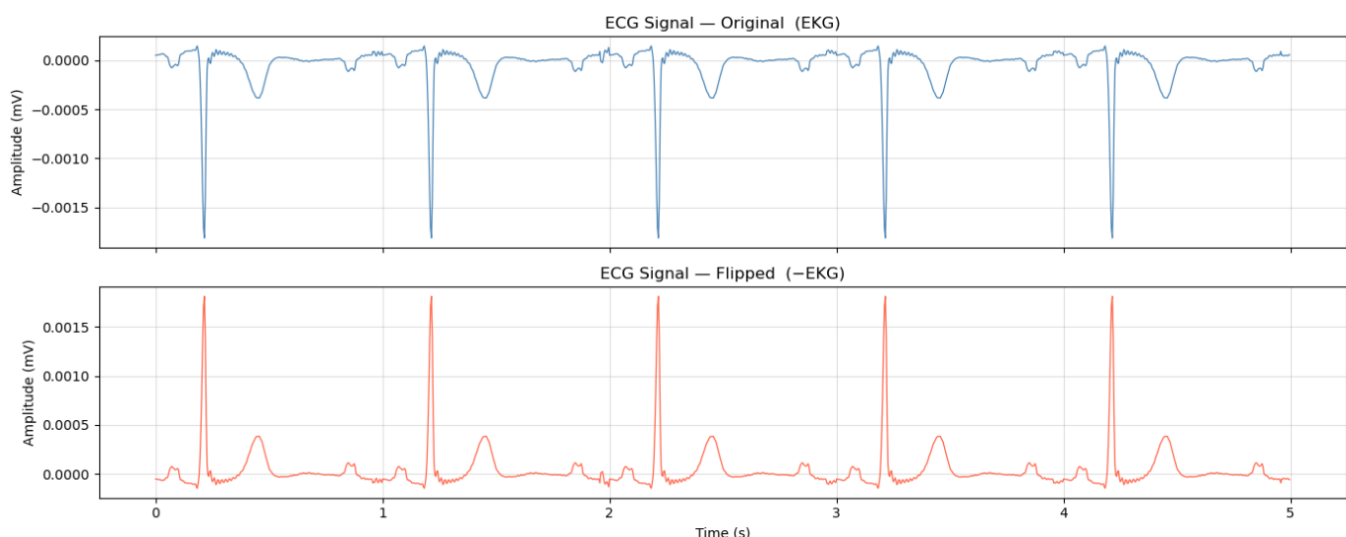
- To ensure reproducibility of results, fixed random seeds were set for both NumPy and TensorFlow.

Data Loading

- In this stage, ECG recordings and their corresponding sleep stage annotations were loaded from the dataset. The data consist of multiple **EDF files** representing ECG signals for different subjects, along with annotation files containing labeled sleep stages.
- Each EDF file was read using the MNE library, and only ECG-related channels (e.g., ECG or EKG) selected for further processing. The raw ECG signal and its sampling frequency were extracted for each subject.
- For each recording, the corresponding annotation file was identified and parsed. **Sleep stage labels were extracted from the annotation data and mapped into five standardized classes: Wake, N1, N2, N3, and REM.**
- Subjects with missing ECG channels or annotation files were excluded to ensure data consistency. The successfully **loaded data** from all subjects were then aggregated into structured arrays, including ECG signals, labels, and sampling frequencies.
- This organized dataset serves as the foundation for subsequent preprocessing and feature extraction stages.

✓ Sampling Frequency (Hz): 256.0
 ✓ Channels: ['EKG', 'EOG-L', 'EOG-R', 'EMG', 'EEG1', 'EEG2', 'EEG3', 'Pres', 'Flow', 'Snore', 'Thor', 'Abdo', 'Leg', 'Therm', 'Pos', 'EKG_Off', 'EOG-L_Off', 'EOG-R_Off', 'EMG_Off', 'EEG1_Off', 'EEG2_Off', 'EEG3_Off', 'Pleth', 'OxStatus', 'SpO2', 'HR', 'DHR']
 ✓ ECG Channels found: ['EKG', 'EKG_Off']

ECG Visualisation — First 5 Seconds



Data Splitting

- To ensure a reliable and unbiased evaluation of the model, the **dataset** was **divided into training, validation, and test** sets prior to any further processing.
- All extracted **ECG epochs, HRV feature vectors, and corresponding labels** were first organized into structured arrays. A stratified splitting approach was then applied to preserve the class distribution across all subsets.
- **The dataset was divided as follows:**
 - 80% for training
 - 10% for validation
 - 10% for testing
- First, **80%** of the data was allocated to the **training set**, while the remaining **20%** was **temporarily grouped together**. This remaining portion was then equally **split** into **validation and test sets**, ensuring that both subsets are representative of the original data distribution.
- This **splitting strategy** guarantees that the model is **trained, validated, and tested** on independent data, providing a fair and realistic evaluation of its performance.

Before augmentation:

```
Train: (10206, 7680) | Labels: Counter({0: 4455, 2: 3293, 4: 1009, 1: 754, 3: 695})  
Val:   (1276, 7680) | Labels: Counter({0: 557, 2: 412, 4: 126, 1: 94, 3: 87})  
Test:  (1276, 7680) | Labels: Counter({0: 557, 2: 412, 4: 126, 1: 94, 3: 87})
```

(Data Splitting)

Data Augmentation Strategy

- To address class imbalance in the dataset, a targeted **data augmentation** strategy was applied exclusively to the **training set**.
- First, the **class distribution** within the **training data** was **analyzed to identify the majority and minority classes**. The class with the highest number of samples was selected as the reference target. The goal of **augmentation** was to **increase the number of samples in all minority classes until they matched the size of the majority class**.
- For each minority class, additional samples were generated by repeatedly applying **augmentation techniques** to **existing ECG epochs**. These augmented samples were created in a cyclic manner, ensuring that all original samples contributed to the generation process.

- Importantly, while the ECG signals were augmented, the corresponding **HRV feature vectors** were kept unchanged to **preserve the physiological consistency** of each sample.
- After generating the required number of augmented samples, the **augmented data** were **combined with the original training data** to form a fully balanced dataset across all classes.
- Finally, the **training dataset** was **shuffled** to **ensure random distribution** of samples during **model training**.
- It is important to note that data augmentation was strictly applied only to the **training set**, while validation and test sets remained unchanged to prevent data leakage and ensure fair model evaluation.

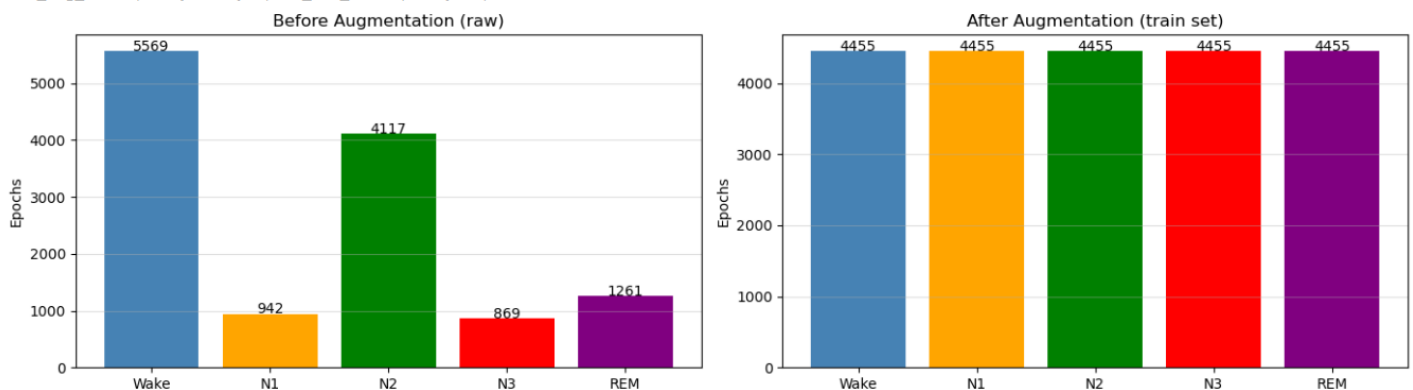
```
Target count per class (= largest class): 4455
Wake : no augmentation needed (4455 epochs)
N1 : 754 -> 4455 (adding 3701 augmented)
N2 : 3293 -> 4455 (adding 1162 augmented)
N3 : 695 -> 4455 (adding 3760 augmented)
REM : 1009 -> 4455 (adding 3446 augmented)
```

After augmentation:

```
Train: (22275, 7680) | Labels: Counter({3: 4455, 4: 4455, 2: 4455, 1: 4455, 0: 4455})
Val: (1276, 7680)
Test: (1276, 7680)
```

Final shapes after normalization:

```
X_ecg_tr : (22275, 7680, 1) X_hrv_tr : (22275, 12)
X_ecg_val: (1276, 7680, 1) X_hrv_val: (1276, 12)
X_ecg_te : (1276, 7680, 1) X_hrv_te : (1276, 12)
```

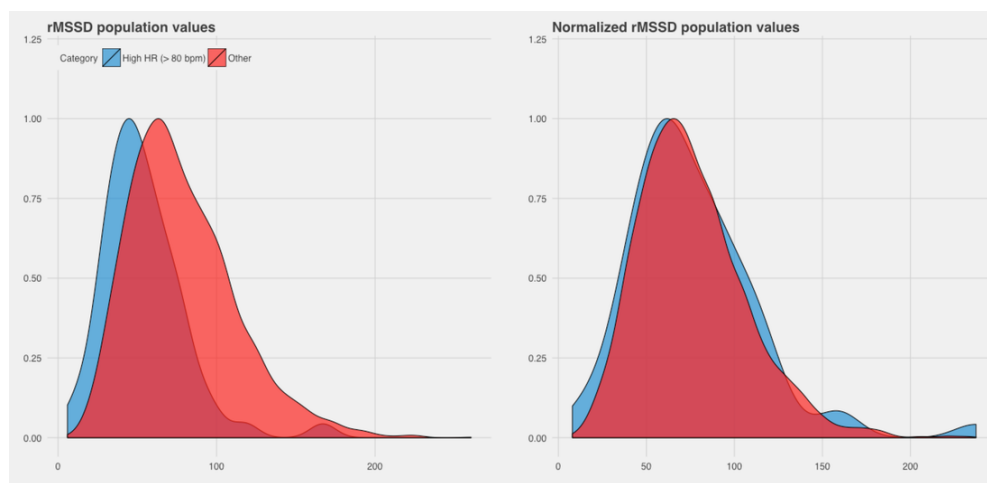


Preprocessing

Signal Normalization

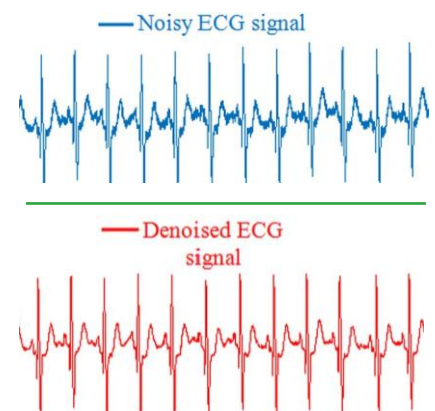
- To standardize the input data and improve model stability, **normalization** was applied to both **ECG signals** and **HRV features**.

- For ECG signals, a robust **normalization method** based on **median** and **interquartile range (IQR)** was used. Each sample was normalized individually by subtracting **the median** and **dividing by the IQR**, making the **process resistant** to **outliers** and **signal artifacts**.
- For **HRV features**, **standard normalization** was applied using **the mean** and **standard deviation** computed from the **training set** only. This approach prevents **data leakage** and **ensures consistent scaling across validation** and **test sets**.
- Finally, all values were clipped within a **fixed range** to **avoid extreme values**, and any **NaN** or **infinite values** were replaced with **valid numerical values** to **ensure numerical stability during training**.



Noise Reduction and Signal Filtering

- To improve the quality of ECG signals, a **bandpass filtering technique** was applied to **remove noise** and **irrelevant frequency components**.
- A **Butterworth bandpass** filter was used with a frequency range of **0.5–40 Hz**. This range was selected to preserve the essential components of the ECG signal while eliminating low-frequency baseline drift and high-frequency noise.
- The **filtering process** was applied to **each ECG signal after loading** and **before further processing**, ensuring that the extracted signal is clean and suitable for analysis.

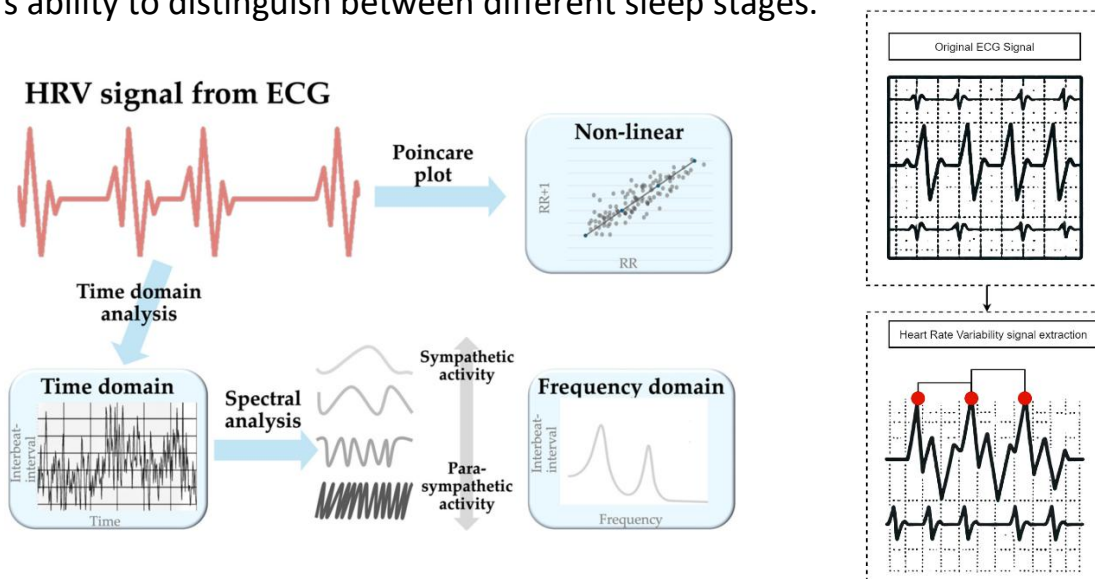


Signal Segmentation

- The continuous ECG signals were **segmented** into **fixed-length time windows** to **align with sleep stage annotations**.
- Each signal was divided into **non-overlapping epochs of 30 seconds**, based on the sampling frequency of the dataset. The **segmentation process** ensured that **each epoch corresponds to a specific sleep stage label**.
- Only epochs with **valid annotations** were **retained for further processing**, ensuring **consistency** between the signal data and their corresponding labels.

Feature Extraction (HRV Features)

- For each segmented ECG epoch, **heart rate variability (HRV) features** were extracted to **capture physiological characteristics of cardiac activity**.
- **R-peaks** were **first detected from the ECG signal**, and **RR intervals** were **computed**. Based on these intervals, multiple features were derived, **including**:
 - **Time-domain features** such as mean RR interval, standard deviation, RMSSD, and pNN50.
 - **Frequency-domain features** including low-frequency (LF) and high-frequency (HF) power, as well as the LF/HF ratio.
 - **Nonlinear features** such as SD1, SD2, and their ratio.
- These features provide complementary information to the raw ECG signal and enhance the model's ability to distinguish between different sleep stages.



Class Weights Strategy

- To handle class imbalance, **class distribution** was first analyzed across the training, validation, and test sets using the corresponding label vectors.
- However, after applying the data augmentation strategy on the training set, **all classes were balanced** to have an equal number of samples. As a result, the need for differential class weighting was eliminated.
- Therefore, **uniform class weights** were assigned during model training, where **each class** was given an equal weight of 1.0. This approach allows the model to learn directly from the balanced dataset without introducing additional bias through weighted loss adjustments.
- This design ensures simplicity in training while maintaining fair representation across all classes.

Class weights (uniform – data already balanced): {0: 1.0, 1: 1.0, 2: 1.0, 3: 1.0, 4: 1.0}
(Class Weights Strategy)

Methodology

Deep Learning Model Architecture

- A **hybrid deep learning architecture** was designed to effectively capture both temporal and statistical characteristics of ECG signals. The model follows a **dual-branch structure**, where **one branch** processes raw ECG signals using a combination of convolutional and attention-based mechanisms, while the **second branch** processes HRV feature vectors using fully connected layers. The **outputs of both branches** are fused to perform final classification into five sleep stages.

Input Shape Definition (ECG Signal + HRV Feature Vector)

- The model accepts **two separate inputs**. The **first input** corresponds to the ECG signal represented as a one-dimensional time-series with shape determined by the length of each epoch. The **second input** consists of a **fixed-length HRV feature vector** extracted from each ECG segment. This **dual-input design** allows the model to leverage both **raw signal dynamics** and **handcrafted physiological features**.

Feature Extraction Layers (CNN / ResNet + Transformer + Dense)

- **Feature extraction** from ECG signals is performed using a **deep hierarchical structure**. Initially, **convolutional layers** are applied to capture low-level temporal patterns. This is followed by multiple residual blocks (**ResNet1D**) with **dilated convolutions** to learn long-range dependencies while preserving gradient flow.
- Subsequently, a **Transformer encoder** is incorporated to model global contextual relationships within the signal using self-attention mechanisms. A **Global Average Pooling layer** is then used to generate a compact feature representation.
- In parallel, the **HRV branch** consists of fully connected (**Dense**) layers that transform statistical HRV features into a higher-level representation. Finally, **features from both branches** are concatenated and passed through additional dense layers to **learn joint representations before classification**.

Activation Functions (ReLU / Softmax)

- The model primarily employs the **Rectified Linear Unit (ReLU)** activation function within convolutional and residual layers to **introduce non-linearity** and **improve training efficiency**. Additionally, **GELU** activation is used within **dense** and **transformer layers** for smoother gradient behavior.
- At the output layer, the **Softmax** activation function is applied to **produce a probability distribution over the five sleep stages**.

Loss Function Design (Focal Loss + Label Smoothing)

- To address class imbalance and improve model robustness on hard-to-classify samples, a composite **loss function** was designed combining **Focal Loss** and **Label Smoothing Cross-Entropy**.
- **Focal Loss** was employed to **focus learning on misclassified and difficult examples by down-weighting easy samples**. A focusing parameter of $\gamma = 3.0$ was used to **strongly emphasize challenging cases**, which is particularly beneficial for minority sleep stages.

- In parallel, **Label Smoothing** with a smoothing factor of **0.1** was incorporated to **reduce model overconfidence** and **improve generalization** by softening hard class boundaries.
- The final loss function is defined as an equal weighted combination (50/50) of both components:
 - **Focal Loss** for hard example mining
 - **Label-smoothed** Cross-Entropy for regularization and stability
- This hybrid formulation improves both class balance handling and generalization capability in multi-class sleep stage classification.

Optimizer Configuration

- The model was optimized using the **AdamW** optimizer, which combines adaptive moment estimation with decoupled weight decay regularization. This helps **improve generalization performance by preventing overfitting while maintaining fast convergence**.
- The optimizer was configured with:
 - Learning rate: 1e-3
 - Weight decay: 1e-4
- **AdamW** was selected due to its stability and effectiveness in training deep neural networks with mixed architectures (**CNN + Transformer + Dense layers**).

Model Compilation

- The final model was compiled using the defined composite **loss function** and **AdamW** optimizer. The evaluation metric used during training was categorical accuracy, which provides a direct measure of classification performance across the five sleep stages.
- This configuration ensures a balanced training process that is **robust** to **class imbalance**, **overfitting**, and **optimization instability**.

Model Summary

- The final model is a **multi-input** neural network that integrates convolutional, residual, and transformer-based components for ECG processing, alongside a dense branch for HRV

features. The architecture concludes with **fully connected layers** and a **Softmax classifier** for multi-class prediction.

- This design enables the model to effectively combine temporal signal analysis with physiological feature representation, resulting in improved performance for sleep stage classification.

Training

Epochs Configuration

- The training process was performed in two stages. In the **first stage**, the model was trained for **10 epochs** using the full learning rate to allow stable convergence of deep feature representations. In the **second stage (fine-tuning phase)**, the model was further trained for an additional **10 epochs** with a reduced learning rate to refine learned representations and improve generalization performance.

Batch Size

- A fixed **batch size of 32** was used during both training phases. This batch size provides a balance between computational efficiency and stable gradient updates, ensuring effective learning across both **ECG signals** and **HRV feature inputs**.

Learning Rate Strategy

- A two-stage learning rate schedule was adopted. In the initial training phase, a learning rate of $1e-3$ was used with the AdamW optimizer to enable fast convergence. During fine-tuning, the learning rate was reduced to $2e-5$ with a lower weight decay ($1e-5$) to allow fine adjustment of pretrained weights and prevent overfitting.

Validation Strategy

- Model performance was continuously monitored using a separate validation set (**10% of the dataset**). Validation metrics, particularly validation loss and validation accuracy, were used to evaluate generalization performance during training. This strategy ensures that the model is assessed on unseen data at every epoch.

Training on Balanced Dataset

- The model was trained on a **class-balanced dataset** obtained after applying augmentation to the training set. This ensured equal representation of all sleep stages (**Wake, N1, N2, N3, REM**), reducing class bias and improving model robustness. Class weights were also defined uniformly, as the dataset was already balanced.

Callbacks and Training Control

- To enhance training stability and prevent overfitting, **multiple callbacks were used**:
 - **ModelCheckpoint**: Used to save the best-performing model based on validation accuracy.
 - **ReduceLROnPlateau**: Automatically reduces the learning rate when validation loss stagnates, improving convergence during fine-tuning.
- In the second training phase, the best model from the first phase was loaded before fine-tuning to ensure continuity and performance improvement.

Evaluation

Test Set Evaluation

- The final trained model was evaluated on an independent test set that was not used during training or validation to ensure unbiased performance assessment. **The best-performing model weights** from the fine-tuning stage were loaded prior to evaluation.
- **Predictions** were generated in a probabilistic manner and converted into class labels using the argmax function across the output **softmax** probabilities.
- **The following evaluation metrics were computed**:
 - **Accuracy**: measures the overall proportion of correctly classified samples.
 - **Balanced Accuracy**: accounts for class imbalance by averaging recall across all classes.
 - **Cohen's Kappa**: evaluates agreement between predicted and true labels beyond chance level.
- In addition, a detailed classification report was generated, providing precision, recall, and F1-score for each sleep stage (**Wake, N1, N2, N3, REM**). These metrics offer a comprehensive assessment of model performance across both majority and minority classes.

Precision, Recall, and F1-Score

- The model performance was further analyzed using precision, recall, and F1-score for each class:
 - **Precision** measures the proportion of correctly predicted samples among all predicted samples for a given class.
 - **Recall** measures the proportion of correctly identified samples among all actual samples of a class.
 - **F1-score** represents the harmonic mean of precision and recall, providing a balanced performance indicator.
- These metrics are particularly important in multi-class imbalanced problems such as sleep stage classification, where some classes may be harder to detect than others.

```
=====
Test Accuracy      : 0.7970 (79.70%)
Balanced Accuracy  : 0.7339
Cohen's Kappa      : 0.7093
=====
```

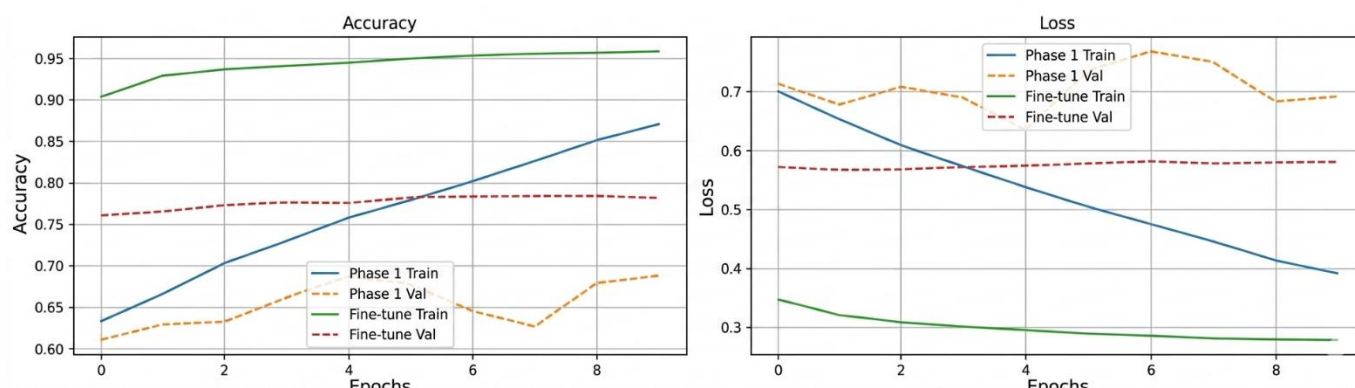
	precision	recall	f1-score	support
Wake	0.95	0.92	0.93	557
N1	0.40	0.52	0.46	94
N2	0.79	0.71	0.75	412
N3	0.62	0.72	0.67	87
REM	0.69	0.79	0.74	126
accuracy			0.80	1276
macro avg	0.69	0.73	0.71	1276
weighted avg	0.81	0.80	0.80	1276

Results

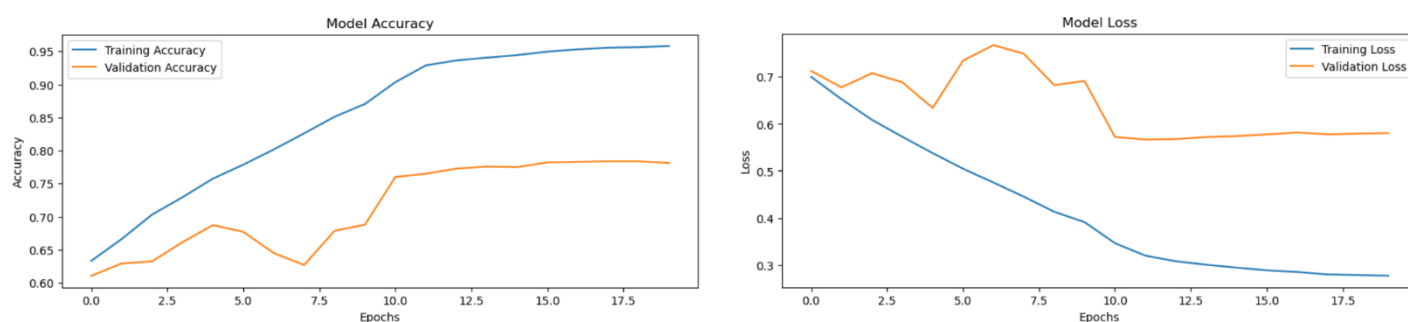
Training and Validation Curves

- The learning behavior of the model was analyzed using training and validation curves for both accuracy and loss across epochs.
 - The **accuracy curves** show the progression of model learning over time for both training and validation sets.
 - The **loss curves** illustrate how the optimization process minimized the objective function during training.
- The comparison between base training and fine-tuning phases demonstrates improved convergence stability and enhanced generalization after learning rate reduction.

- These curves also indicate that the model does not exhibit severe overfitting, as validation performance closely follows training performance.



(Accuracy and Loss Curve)



(Accuracy and Loss Curves After Merging Fine-Tuning with Training)

Class Distribution Analysis

- The **class distribution** of the test set and model predictions was analyzed to **evaluate prediction balance**.
- The **test set distribution** confirms that **all sleep stages are represented in evaluation**. The predicted distribution shows how the model assigns labels across classes, providing insight into potential bias or over-/under-prediction of certain stages.
- This comparison confirms that the model maintains a relatively balanced prediction behavior across all classes.

Test set distribution:

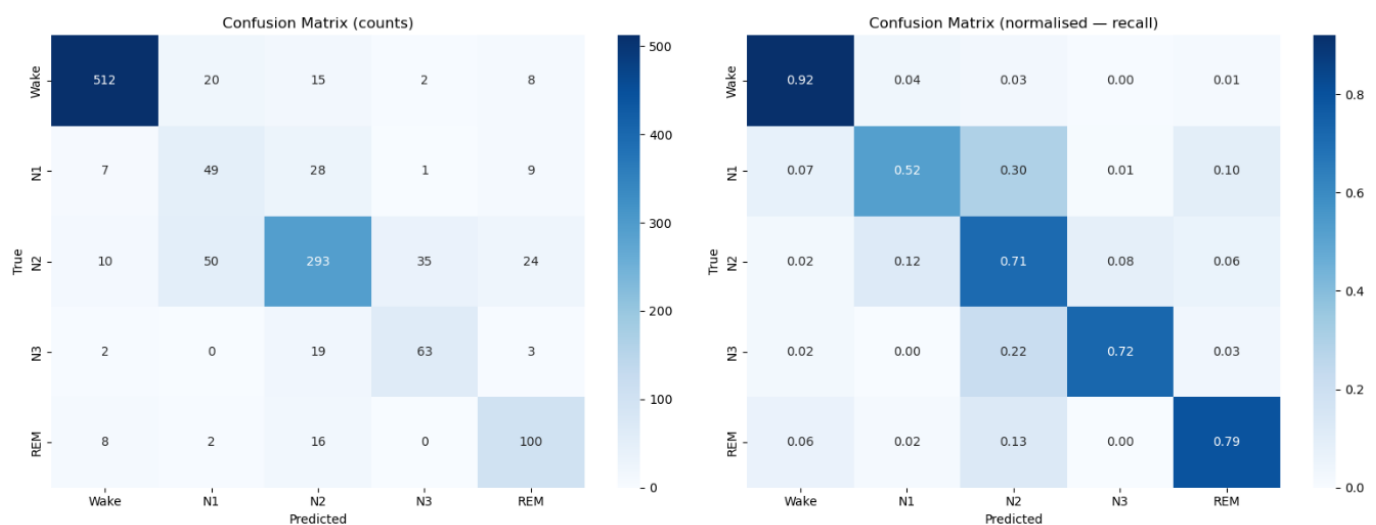
Wake	: 557 samples
N1	: 94 samples
N2	: 412 samples
N3	: 87 samples
REM	: 126 samples

Prediction distribution:

Wake	: 539 predicted
N1	: 121 predicted
N2	: 371 predicted
N3	: 101 predicted
REM	: 144 predicted

Confusion Matrix Analysis

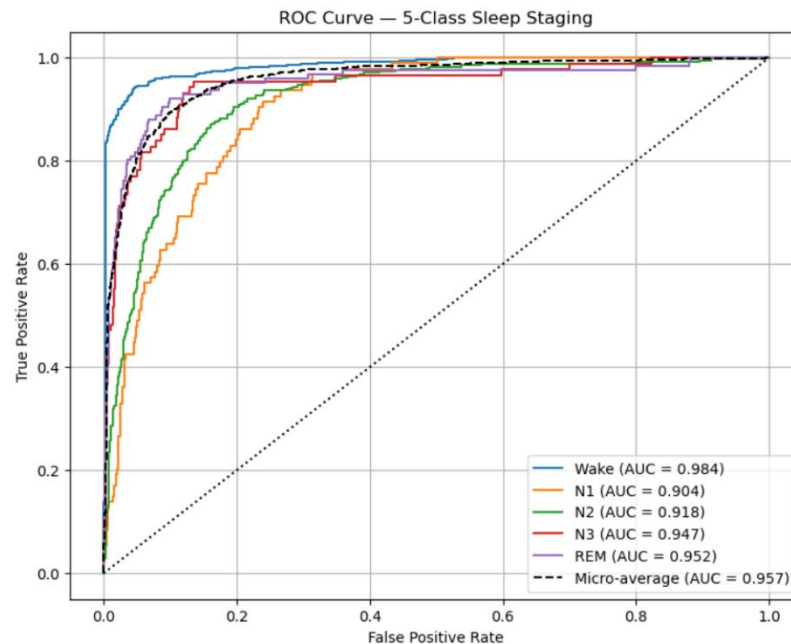
- A **5×5 confusion matrix** was used to evaluate class-wise prediction performance.
- **Two representations were analyzed:**
 - **Raw confusion matrix**, showing the absolute number of correct and incorrect predictions.
 - **Normalized confusion matrix**, representing recall-based performance for each class.
- The results highlight the model's ability to correctly classify most sleep stages, while also revealing confusion patterns between physiologically similar stages such as N1 and N2 or N2 and N3.



ROC Curve Analysis

- **Receiver Operating Characteristic (ROC)** curves were generated using a one-vs-rest strategy for all five classes.
- Each class-specific ROC curve illustrates the trade-off between true positive rate and false positive rate across different classification thresholds. The **Area Under the Curve (AUC)** was computed for each class to **quantify discriminative performance**.
- Additionally, a **micro-average ROC curve** was calculated to **provide an overall performance measure across all classes**.

- High AUC values across classes indicate strong separability and robust classification capability of the proposed model.



Discussion

Model Strengths

- The proposed model demonstrates several key strengths resulting from its hybrid architecture and carefully designed preprocessing pipeline. First, the integration of a dual-input structure combining raw **ECG signals** with **HRV feature vectors** enables the model to learn both deep temporal patterns and handcrafted physiological descriptors, improving overall discriminative power.
- Second, the use of a **ResNet1D backbone** with dilated convolutions allows efficient extraction of both local and long-range dependencies in ECG signals, which is critical for sleep stage classification. The addition of a Transformer encoder further enhances the model's ability to capture global temporal relationships within the signal.
- Third, the application of biologically realistic **data augmentation** significantly improves model generalization by exposing the network to controlled variations in ECG morphology. Combined with **class-balanced training**, this reduces bias toward majority classes and improves minority class recognition.
- Finally, the use of a composite **loss function** (Focal Loss + Label Smoothing) and the **AdamW** optimizer enhances robustness against class imbalance and improves convergence stability.

Model Limitations

- Despite its strong performance, the proposed approach has some limitations. The model relies on a relatively small subset of subjects, which may limit generalization to larger and more diverse populations.
- Additionally, the quality of **HRV features** depends on accurate R-peak detection, which may be affected by noise or signal artifacts in real-world settings. Any errors in peak detection can propagate through the **feature extraction pipeline**.
- The model also requires relatively high computational resources due to the combination of deep convolutional layers and Transformer blocks, which may limit its deployment in **low-power** or **real-time wearable devices**.
- Finally, although data augmentation improves robustness, it does not fully replace the need for larger and more diverse real-world datasets.

Conclusion

Study Summary

- This study presented a hybrid deep learning framework for automatic sleep stage classification using **ECG signals** and **HRV features**. The proposed system integrates a ResNet1D-based convolutional backbone, a Transformer encoder for temporal modeling, and a parallel HRV feature branch.
- A comprehensive **preprocessing** pipeline was developed, including bandpass filtering, signal segmentation, **HRV feature extraction**, normalization, and class-balanced augmentation. The model was trained using a combination of Focal Loss and Label Smoothing with the **AdamW** optimizer to enhance robustness and generalization.
- Experimental results demonstrate that the proposed approach achieves strong performance in classifying five sleep stages (**Wake, N1, N2, N3, REM**), with stable training behavior and good generalization on unseen data.

Future Improvements

- Future work can focus on improving model generalization by training on larger and more diverse datasets collected from multiple sources. Incorporating additional physiological signals such as EEG or respiration could further enhance classification accuracy.
- From a modeling perspective, **lightweight architectures** could be explored to enable real-time deployment in **wearable** or **embedded systems**. Additionally, self-supervised or contrastive learning approaches may reduce dependence on labeled data.
- Finally, improving R-peak detection robustness and exploring adaptive augmentation strategies could further enhance system reliability in real-world clinical applications.



(QR Code of GitHub)